EthOzone: Private Accesses to the Ethereum Blockchain*

Afonso Tinoco[1]

[1]Oblivious Labs Inc., afonso.tfaria21@gmail.com

Abstract

We will design and build a low-latency and high-throughput oblivious database called EthOzone that enables private retrieval of Ethereum's blockchain data. A client can query the value associated with some address in Ethereum's global state, and at the same time fetch the corresponding Merkle proof. We will provide two alternative instantiations of EthOzone: one based on "trusted hardware + Oblivious RAM (ORAM)", and one based on Private Information Retrieval (PIR), thus offering different tradeoffs between performance and trust assumptions. Our team currently maintains the fastest implementations of both ORAM and PIR schemes. This project will focus on tailoring our techniques to Ethereum's Merkle Patricia Tree to deliver maximal performance, while providing the option of retrieving the Merkle proofs along with the data values.

1 Objectives

Objectives. We will design and develop an *oblivious database* that supports queries to the Ethereum full state as well as the blockchain logs, without revealing the address or transaction being queried. We will provide two alternative instantiations, offering a tradeoff between performance and trust assumptions:

1. *Instantiation based on ORAM + TEE.* The client sends the queries in encrypted form to the Trusted Execution Environment (TEE) on the server. The server will maintain an up-to-date copy of Ethereum's blockchain in encrypted and ORAM'ed format. When the server's TEE fetches the relevant data blocks from the storage medium (including untrusted RAM, local or networked drives), the locations accessed will be randomized by the Oblivious RAM (ORAM) algorithm. The ORAM algorithm provides mathematical guarantee that the server learns nothing about the intended locations.

   We expect that this instantiation should deliver the fastest performance, at the price of having to trust the TEE. Specifically, Signal's large-scale deployment of ORAM [sig] provides a testimonial of the scalability of ORAM to blockchain-scale datasets. As mentioned in more detail in Section 6.1, our team maintains an efficient ORAM implementation that is one to two orders of magnitude faster than the implementations of Signal and Meta.
2. *Instantiation based on PIR.* Whenever the client wants to query the data residing at some address, its local machine runs the client algorithm of the Private Information Retrieval (PIR) scheme, which encodes the desired address as a set of random-looking queries that leak no information. The server runs the PIR's server algorithm, and sends back the answers to these queries. The client then runs a decoding algorithm which returns the data at the desired address.

*EthOzone, also written as EthO3, is short for "Ethereum: an Optimal Oblivious Oracle", and serves as a protective layer for Ethereum.

This instantiation will have relatively slower performance in comparison with the TEE+ORAM solution. However, security is cryptographically guaranteed and does not rely on any trusted hardware. Moreover, by leveraging suitable preprocessing techniques, we expect to achieve orders of magnitude performance improvement over state-of-the-art classical PIR scheme (e.g., SimplePIR [HHCG+23]).

For both instantiations, we will enable an *authenticated retrieval mode*, that is, the query will fetch the corresponding Merkle proof along with the data that resides at some address. The Merkle proof can then be provided to a downstream consumer, e.g., to prove correctness of contract execution to a private block builder running in TEE, or to the prover of a zero-knowledge scheme.

Success criteria and anticipated performance for Ethereum. Our success criteria include achieving low-latency, high-throughput query performance for the full Ethereum state, while maintaining end-to-end privacy and delivering a thoroughly documented solution to be open sourced under the MIT license. We anticipate the following ballpark performance — the estimates are extrapolated using our current ORAM and PIR implementations available at `https://github.com/obliviouslabs/oram` and `https://github.com/wuwuz/Piano-PIR-new` :

- Our TEE+ORAM implementation: we expect the performance to be dominated by the cost of accessing external memory (SSD). Assuming a consumer-grade SSD, and 8x concurrent threads, we expect $10\mu s$ per query for `getAccount` operations, and 40μ per query for a `getStorage` operations. For `getProof` operations (on an account + memory address), we expect $100\mu s$ per query. For all scenarios, the database update time should be similar to the query time, and we expect at least an order of magnitude improvement over Signal's ORAM implementation [sig] — see Section 6.1 for our current capabilities.
- Our PIR-based implementation: we expect the performance to be dominated by the computational overhead (as opposed to network transmission). Assuming 2 concurrent threads, we expect 17ms to 30ms computational overhead for a `getAccount` operation and 30ms to 100ms computational overhead for a `getStorage` operation. For a `getProof` operation (on an account), we expect 75ms to 100ms computational overhead assuming 8 concurrent threads. For all scenarios, we expect at least two orders of magnitude performance improvement over a state-of-the-art classical PIR scheme such as SimplePIR [HHCG+23].

Threat model and architectural considerations. We now discuss the threat model, trust assumptions, and architectural considerations in more detail.

- **EthOzone via TEE + ORAM:** For our TEE+ORAM solution, we assume that the server machine is fully under the control of the adversary; however, the adversary cannot break into the TEE. In particular, the adversary may have full control of the operating system, can launch timing side-channel attacks, and can perform physical attacks such as cold-boot style attacks.
- **EthOzone via PIR:** We will provide solutions for the 1-server and 2-server settings. For the 1-server setting, the storage server is in the full control of the adversary. For the 2-server setting, we assume that the two servers do not collude. We will additionally offer a "defense-in-depth" solution that uses TEE + ORAM to implement one of the two servers called the preprocessing server. In this case, it is sufficient to assume that that either the 2 servers do not collude, or the TEE on the preprocessing server is not compromised.

For a TEE architecture, one naïve solution without ORAM is to store the entire Ethereum's state and blockchain data inside the TEE's protected memory region. However, this solution suffers from several drawbacks in terms of scalability and security:

1. First, today's Ethereum full state and logs (about 1.4TB) cannot fit within Intel SGX's protected memory region (ranging from 128MB to 512GB depending on processor model). Although we could instead use Intel TDX which allows the protected memory region to be as large as the RAM, this would come with a significant security tradeoff, since TDX,

designed for running VMs inside the TEE, would reintroduce the entire OS back into the trusted computing base (TCB). In our work, we will deliver implementations for both SGX and TDX — our existing oblivious key-value store implementation works on both platforms.

2. Fitting the entire data into TEE is incompatible with efforts of building distributed data availability layers for Ethereum — in this setting, the storage may be distributed across multiple servers, and the network traffic would leak the access patterns.
3. Even accesses within the TEE are prone to cache-timing attacks. Numerous works have shown that a co-resident VM can observe memory accesses within the TEE through cache-timing attacks [RTSS09, DMWS12]. Our oblivious database protects against such cache-timing attacks.

2 Outcomes

Maximum Extractable Value (MEV) is a major challenge faced by the Ethereum community today. Because miners or block builders can front-run or back-run users' transactions, they can make risk-free profit at the expense of users. Recent research has also shown that MEV can cause centralization effects in Ethereum's eco-system [YNZ25].

To mitigate MEV, the Ethereum community has dedicated significant efforts towards developing new architectures for private order flow, encrypted mempools, or private block building. All of these efforts aim to create a architecture where the transactions remain confidential until the block has been assembled. No matter whether the private block building is accomplished through TEE or cryptography (e.g., multi-party computation or fully homomorphic encryption), a key challenge is to obfuscate the accesses to blockchain data as the (encrypted) transactions are being speculatively executed. In particulary, accesses to blockchain data leak highly sensitive information such as the crypto-asset being traded, and the transactional counterparties.

Our proposed work will solve this challenge by providing low-latency, high-throughput accesses to Ethereum's full state and logs. Our solution lays the groundwork for secure blockchain interactions, specifically by allowing decentralized applications to query Ethereum state in a manner that preserves query confidentiality. Besides private block building, our solution also enables many other applications in the blockchain space, such as privacy-preserving lightweight clients, and privacy-preserving transactions and smart contracts.

Last but not the least, we will benefit the Ethereum eco-system by open sourcing our code under the MIT license.

3 Scope

The anticipated deliverables include:

- *Open-source implementations:* we will deliver two alternative implementations of EthOzone, one via TEE+ ORAM, and one via PIR. The implementations will also come with detailed documentation.
- *Academic publications:* we will present our findings in peer-reviewed academic publications, detailing the algorithms, proofs, practical optimizations, and performance evaluations.

4 Related Work

4.1 Algorithmic Background on Oblivious RAM

We now review some technical background on oblivious RAM (ORAM) [SvDS+13, WCS15].

Main data structure. An ORAM scheme allows one to access a logical memory array $\mathsf{arr}[1 \ldots N]$ without leaking the index being accessed. To achieve this, the core data structure is a binary tree. Each node in the tree is a bucket containing a combination of *real* blocks and *filler* blocks. The real blocks are encryptions of actual data, whereas the filler blocks are encryptions of zero, and they are needed for security. The root bucket has super-logarithmically many slots, whereas all other buckets have only 4 slots (the number of slots can be adjusted depending on the eviction algorithm to be described later).

Path invariant. At any point of time, every logical index addr is assigned to a random path in the tree, i.e., a path from the root to some random leaf node. This means that the encrypted block at addr must reside somewhere on the path.

Operations. Reading or writing a block involves two steps as mentioned below:

1. *Read and remove.* To fetch a block at logical index addr, the client accesses the path associated with addr. Once the block is accessed, it is removed from the original position, and its new value is written back to the root bucket.
2. *Evict.* To prevent the root bucket (or any bucket) from filling up, the client performs some maintenance operations (called *eviction*) on the path just visited. Specifically, the client rearranges the blocks on the path, packing them as close to the leaf level as possible while still respecting the path invariant (i.e., no block can be moved away from its designated path).

In all operations, whenever the client reads a physical block, it always rewrites back a reencrypted block no matter whether the data has changed or not.

Recursion. Based on the above algorithm, there is only one gap remaining: how do we know which path the desired block resides on? This is accomplished through a recursive data structure. Imagine that we have a *position map* that stores the path identifier of every block. If each block has size B, then the position map can be stored using a total of $O(N/B)$ blocks. We now recurse and store the position map in a smaller ORAM, until eventually the position map (of the position map of the position map ...) is constant size.

Efficiency. Assuming the ORAM client caches the root bucket, then the total cost of an access is one path at each recursion level. The number of recursions is $O(\log_B N)$, and the path length is $O(\log N)$. So the total cost of an access is $O(\log_B N \cdot \log N)$. In practice, the $\log_B N$ part is typically 2 or 3. So the algorithm enjoys logarithmic performance scaling in practice, comparable to the overhead of accessing data stored in a database. We stress that ORAM itself does not involve any cryptographic computations. Thus, when used in conjunction with TEE, we can enjoy almost native performance speeds — see Section 6.1 for concrete performance numbers.

4.2 Algorithmic Background on PIR

Classical PIR: linear computation per query. Private Information Retrieval (PIR) allows clients to fetch entries from a large, public database (e.g., the Ethereum blockchain), without leaking which entry is accessed. The main difference between PIR and ORAM is that in ORAM, the server must store a separate (encrypted) copy of the database for each user, whereas in PIR, the server need not store per-client state. For this reason, using ORAM as a PIR scheme requires having a TEE on the server machine which is a single trusted client acting on behalf of all users.

Classical PIR schemes (without preprocessing) [CGKS95, HHCG+23, MW22] all suffer from $\Omega(n)$ cost per request where n is the database size. Beimel et al. [BIM00] proved that this limitation is inherent: intuitively, if there is some location that the server(s) need not visit during some query, then the client cannot be requesting that location. The linear computation barrier makes PIR completely impractical for blockchain-scale datasets. At the 2023 Confidential Computing Summit, Signal's tech lead Rolfe Schmidt highlighted this challenge in his talk,

noting that if Signal were to use a state-of-the-art classical PIR scheme for private contact discovery, their annual computing cost would amount to **hundreds of millions of dollars**.

A paradigm shift towards preprocessing PIR. A line of recent works [BIM00, CK20, LP23, KCG21, ZPSZ24, GZS24] showed that preprocessing can allow us to overcome this linear barrier for PIR. Specifically, our team's own prior work Piano [ZPSZ24] was among the very first to make preprocessing PIR practical — see Section 6.2 for more details.

4.3 Gaps to Address

The main gaps to address include:

1. Plain ORAM and PIR are designed for a memory-array abstraction. We will customize the algorithms for Ethereum's Merkle Patricia Tree (MPT).
2. We will develop algorithmic improvements to support efficient batch queries.
3. We will rely on differential obliviousness (DO) to further improve the efficiency while still achieving mathematically quantifiable security.
4. For our PIR-based solution, we will rely on ORAM+TEE to realize the PIR's preprocessing phase to achieve "defense in depth".

See Section 7 for a more detailed explanation of the challenges and our proposed methodology.

5 Team

The proposed project will be conducted at Oblivious Labs Inc. Oblivious Labs Inc. is committed to open-source, and providing libraries and toolchains for oblivious data accesses as a public good. So far, **all software artifacts and algorithms produced by Oblivious Labs have been open sourced under the MIT license** — see our git repositories at `https://github.com/obliviouslabs`.

PI: Afonso Tinoco. Tinico is a Ph.D. candidate on leave from CMU. He is a co-founder and lead research engineer at Oblivious Labs, Inc. Tinoco has had extensive prior experience with both oblivious algorithms and trusted hardware [TGS23, GWC+25], and has been a key contributor to Oblivious Lab's ORAM open source [ola].

Tinoco was the captain and co-founder of Lisbon cybersecurity CTF team (2015-2023). Tinoco has demonstrated exceptional problem-solving skills and algorithmic proficiency through his participation in both CTF and competitive programming international finals, including CSAW CTF (2018, 2019, 2021, 2023 (2nd place)), European CyberSecurity Challenge (2021), Hack In The Box Pro CTF (2021), RuCTF (2018, 2019), VolgaCTF (2017), ACM ICPC Regional SWERC (2015, 2016), and IOI (2012, 2013, 2014).

Key personnel: Junxi Song. Song is a Ph.D. student currently on leave from CMU. She is a co-founder and lead research engineer at Oblivious Labs. During her Ph.D., Song worked on ORAM research, and she has been developing a high-performance ORAM library in Rust at Oblivious Labs. Previously, she co-founded and served as CTO of HackQuest, one of the largest Web3 developer education platforms, where she architected and scaled its core product, delivering extensive tutorials and courses on Layer 1 and Layer 2 blockchain development and security.

Key personnel: Mingxun Zhou. Zhou is a Ph.D. candidate at CMU scheduled to graduate in April. He will join Oblivious Labs either full-time or as a consultant on the proposed project. Zhou's Ph.D. research has focused on making PIR practically efficient. In particular, his work Piano [ZPSZ24] and subsequent works [GZS24, ZSF25] have fuelled a new paradigm shift in the practical landscape of PIR — see Section 6.2 for more details. Zhou is the recipient of a Cylab Presidential Fellowship, and multiple prestigious scholarships and fellowships during his

Table 1: Query latency: comparison with other state-of-the-art implementations. The performance numbers bellow are in **microseconds**. The comparison is performed on a machine with an Intel Xeon Platinum 8352S processor and an Intel P5500 SSD.

data size	Oblix	Signal	Meta	MobileCoin	**Our prior work**
2^{14}			28	80	4.97
2^{20}	14000	76	460	120	7.84
2^{28}	24000	260	-	250	40
2^{30}	32000	316	-	-	50

undergraduate studies at PKU. He won gold and silver medals in the National Olympiad in Informatics (NOI).

6 Background

6.1 Existing Capability: Open-source Oblivious Key-Value Store

At Oblivious Labs, we have developed an oblivious key-value store that is optimized for hardware enclaves. We have made our implementation open source under MIT license at `https://github.com/obliviouslabs/oram`. To the best of our knowledge, ours is the fastest oblivious key-value store implementation available today.

As shown in Table 1, our implementation achieves **6.3× to 9.7×** speedup over Signal[1] even when we *allow Signal to have up to 550GB protected enclave memory*, but *restrict our own implementation to only 64GB protected enclave memory*. We achieve **5.6× to 58.6×** speedup over Meta's implementation on smaller databases, and the speedup will only become more significant for larger datasets. Note also that Meta's implementation supports only a memory-array abstraction, and modifying their implementation to support key-value queries would incur another $\sim 10\times$ overhead.

Besides query latency, we also measured the throughput of our implementation under a batched setting. Assuming a 4096 batch size, our existing implementation supports **200K queries/s** on a single machine for a database of size 2^{28}. In comparison, Signal's implementation supports only 35.7K queries/s for the same data size.

Our oblivious key-value store is based on our Enigmap paper [TGS23]. We stress that since the publication of the paper, we have implemented numerous algorithmic improvements that have led to 10× to 20× performance improvement over the original implementation evaluated in the Enigmap paper. The above table reflects the performance of our latest open-source version.

6.2 Existing Capability: PIR

Our recent works Piano [ZPSZ24] and subsequent results [GZS24, ZSF25] have played a significant role in fuelling a paradigm shift in the practical landscape of PIR. Specifically, we showed that with preprocessing, we can design a surprisingly simple PIR scheme that makes use of only Pseudorandom Functions (PRFs) and no public-key cryptography. Piano achieves $\widetilde{O}(\sqrt{n})$ cost per query assuming that the client has $\widetilde{O}(\sqrt{n})$ space. Our evaluation shows that Piano outperforms state-of-the-art classical PIR schemes such as SimplePIR [HHCG+23] by **two orders of magnitude** on moderately sized databases (e.g., 100GB). Since this improvement is asymptotical in nature, we expect an even more dramatic speedup over classical PIR for larger data sizes.

[1]The comparison uses Signal's in-production implementation as of March 2025.

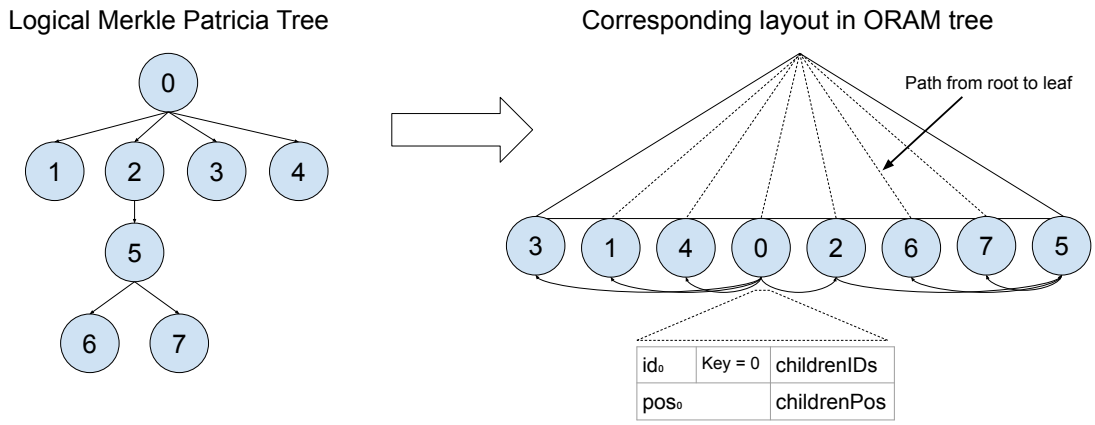


Figure 1: Each node in the logical MPT (henceforth called an MPT node) lives on some random path in the ORAM tree — for convenience, we draw the node at the bottom of the path it belongs to. Each MPT node stores the path identifiers of its children in the MPT. This way, once we fetch an MPT node, we immediately know which path to visit to fetch any of its children.

We have released open-source code on our practical PIR implementations[2], available at https://github.com/wuwuz/Piano-PIR-new.

7 Methodology

7.1 EthOzone via TEE + ORAM

Tailoring ORAM for Ethereum's Merkle Patricia Tree. Plain ORAM provides a memory array abstraction. To support oblivious accesses to Ethereum's blockchain data, one main challenge is to tailor ORAM to Ethereum's Merkle Patricia Tree (MPT) data structure. The naïve way of flattening the MPT into an array and then using ORAM will incur an extra logarithmic factor. To avoid this log-factor blowup, a more efficient approach is to align the MPT's logical tree structure with the ORAM's recursion, a technique used for realizing efficient oblivious data structures [WNL+14]. In our prior work, we leveraged these techniques to provide an efficient oblivious key-value store, where the logical tree structure is an AVL tree — see Section 6.1 for concrete performance numbers.

In the proposed work, we aim to tailor our techniques for Ethereum's MPT to maximize performance — see Figure 1. Specifically, we will adapt our oblivious data structure techniques to support operations specific to Ethereum's MPT, such as compressing paths where nodes have no siblings, as well as the reverse decompression operation.

Additionally, we want to support an authenticated mode of retrieval, where a user can simultaneously request the corresponding Merkle proof when retrieving the value associated with some address.

Using differential oblivious techniques to further improve performance. Since the paths have different lengths in Ethereum's MPT, achieving full obliviousness requires padding the number of accesses to the maximum length possible, which will incur additional overhead. We will explore the use of differential obliviousness (DO) techniques [CCMS22] to reduce the overhead caused by padding. Instead of always padding to the worst case, we will add some noise to the number of accesses in a way that satisfies mathematically quantifiable privacy.

[2]This work was performed during Zhou's Ph.D. studies at CMU.

The use of DO techniques also opens up new opportunities for caching, which can further improve efficiency. Some forms of caching, e.g., caching frequently accessed locations, may also leak information through the number of disk accesses. We plan to defend against such leakage also using DO techniques.

Efficient batched queries. Popular Ethereum full node implementations such as Geth and reth support batch storage queries. We will further improve the efficiency for batch queries. Specifically, known ORAM schemes are friendly to parallelization, first shown in a line of work called Oblivious Parallel RAM (OPRAM) [BCP15, HCS17], with a practical instantiation being Snoopy [DFD+21]. Intuitively, ORAM essentially routes each request to a random partition of the tree, and the load received by each subtree is roughly balanced with high probability. This natural load-balancing behavior facilitates parallelization in a natural manner. In the proposed work, we will achieve parallelization in two ways: 1) by leveraging multi-threading on a single machine, and 2) by randomly distributing requests to multiple servers equipped with TEE.

7.2 EthOzone via Preprocessing PIR

Tailoring PIR for Ethereum's Merkle Patricia Tree. Plain PIR provides a memory-array abstraction. We will work on tailoring PIR techniques to Ethereum's Merkle Patricia Tree (MPT). A naïve but inefficient approach is to flatten the MPT into a memory array and run PIR on this flattened array. Traversing a path in the MPT will turn into logarithmically many *sequential* PIR queries. To improve efficiency and avoid the logarithmically many sequential rounds, our preliminary idea involves the following two steps:

1. *Develop a KV-PIR.* First, we will adapt our current PIR implementation into a PIR that supports key-value queries (henceforth called KV-PIR), using an approach similar to Celi and Davidson [CD24]. The high-level idea is to use a Cuckoo-hash table to hash the keys (from a large universe) to a memory array whose size is proportional to the number of actual items, not the universe size. We will run a PIR scheme on top of the hash table, and fetching any key requires making two PIR queries over this hash table.
2. *Separate instances of KV-PIR for each level of the MPT.* We will put each level of the MPT into a separate instance of KV-PIR. All instances can be executed in parallel. The nodes that are part of a compressed path in the MPT may introduce filler entries for the corresponding level's KV-PIR instance.

Efficient batched queries. We will adapt our current PIR implementations to support efficient batched queries. To achieve this, we will adapt the techniques of Angel et al. [ACLS18] originally suggested for classical PIR to fit preprocessing PIR. For example, consider the special case of having roughly $\sqrt{n}$ queries. The naïve approach is to run $\sqrt{n}$ PIR queries on a database of size n. However, if we hash the database into roughly $\sqrt{n}/\mathsf{poly}\log(n)$ bins, then each bin receives roughly $\widetilde{O}(\sqrt{n})$ database entries, and $\mathsf{poly}\log(n)$ queries. Therefore, it suffices to run $\mathsf{poly}\log(n)$ PIR queries on smaller databases of size only $\widetilde{O}(\sqrt{n})$.

Support for dynamic databases. Our current implementation works for a static database. Since Ethereum's state is constantly evolving, we will research new techniques to efficiently support a dynamic database. For example, one idea is to leverage a hierarchical data structure [BS80] which provides a generic way of converting a static data structure into a dynamic one. In our proposed work, we will explore how to optimize the hierarchical data structure for private retrieval of Ethereum's blockchain. Our approach is inspired by Kogan and Corrigan-Gibbs [KCG21], but we will base on a more efficient underlying preprocessing PIR than Kogan and Corrigan-Gibbs [KCG21].

New algorithmic techniques for concrete performance improvements. Since our Piano work, various works [HPPY25, WR25], including our own [GZS24, GZSP25] have achieved further asymptotical improvements in several dimensions, e.g., client computation, client storage,

or amount of cryptographic computation. While these works introduce elegant new techniques from a theoretical perspective, their concrete performance is not yet on par with Piano. In the proposed work, we will explore how to leverage these ideas but in new ways that lead to concrete performance improvements.

Finally, the DO techniques used to improve the performance in our ORAM-based implementation (see Section 7.1) also apply to the PIR setting.

Combining PIR and ORAM for defense in depth. As mentioned, preprocessing PIR allows us to overcome the linear computation barrier pertaining to classical PIR. For our prior work Piano, in a single-server setting, the preprocessing consumes $\sqrt{n}$ amortized bandwidth per query. By introducing an additional non-colluding server that handles the preprocessing operations, the (amortized) preprocessing bandwidth can be reduced to $\widetilde{O}(1)$ per query.

In the proposed work, we will leverage TEE + ORAM techniques to implement the preprocessing server. This provides a notion commonly called "defense in-depth". Security is guaranteed as long as one of the following assumptions is respected: 1) the preprocessing server does not collude with the online server; or 2) the preprocessing server fails to tamper with its TEE.

8 Timeline

Task 1: Private retrieval through TEE + ORAM

- **Milestone 1.a: deliver a single-threaded oblivious database for the Ethereum blockchain.** We will begin by developing a single-threaded version of the oblivious database. The oblivious database will support the full spectrum of queries handled by popular Ethereum full nodes. We will provide implementations that work for both SGX and TDX.

  Expected number of hours. 72 personnel-hours.

  Budget. $5108.4 based on a $70.95 hourly rate (not including fringe benefits).

- **Milestone 1.b: parallelism through multi-threading.** Next, we will rely on multi-threading to support batch queries on a single machine.

  Expected number of hours. 72 personnel-hours.

  Budget. $5108.4 based on a $70.95 hourly rate (not including fringe benefits).

- **Milestone 1.c: a distributed oblivious database.** To support larger batches, we will build a distributed oblivious database that distributes a batch of queries to random servers. Each server will manage a partition of the ORAM tree data structure using the parallel ORAM in Milestone 1.b.

  Expected number of hours. 71 personnel-hours.

  Budget. $5037.45 at a $70.95 hourly rate (not including fringe benefits).

Task 2: Private retrieval through PIR

- **Milestone 2.a: deliver a PIR-based oblivious database for the Ethereum blockchain.** We will deliver an alternative implementation of an oblivious database using PIR. The main technical challenge is to adapt our current preprocessing PIR implementation to Ethereum's MPT — see Section 7.2 for more details.

  Expected number of hours. 72 personnel-hours.

  Budget. $5108.4 based on a $70.95 hourly rate (not including fringe benefits).

- **Milestone 2.b: efficient batch queries.** We will add support for efficient batch queries using techniques described in Section 7.2.

Expected number of hours. 72 personnel-hours.

Budget. $5108.4 based on a $70.95 hourly rate (not including fringe benefits).

- **Milestone 2.c: use ORAM + TEE to realize the PIR's preprocessing phase.** We will use ORAM + TEE to realize the PIR's preprocessing phase, thus achieving "defense in depth".

  Expected number of hours. 71 personnel-hours.

  Budget. $5037.45 at a $70.95 hourly rate (not including fringe benefits).

9 Budget

Requested Grant Amount and Allocation

The total requested grant amount is **$38,783.60** , which will be allocated as follows:

Direct Costs

- **Principal researchers salaries:**
  - **Tinoco**: $12,771.00 (180 hours at $70.95/hr)
  - **Song**: $12,771.00 (180 hours at $70.95/hr)
  - **Zhou**: $5,676.00 (70 hours at $70.95/hr)
  - **Total Direct Labor**: $31,218.00
- **Fringe benefits (taxes, health, PTO, holidays, etc.):**
  - **Tinoco**: $2,282.18
  - **Song**: $2,282.18
  - **Total Fringe Benefits**: $4,564.36
- **Hardware costs**
  - **Server Rental**: $600.00 (2 servers for 6 months at $50.00/month)

Indirect Costs

We will charge 6.60% of indirect cost, which consists of book-keeping and payroll, accountant services, legal services, HR/grant management support, and virtual office & co-working space.

Total Indirect Cost: $2,401.24

Summary

- **Total Direct Costs (Labor + Hardware + Software + Fringe)**: $36,382.36
- **Indirect Costs (G&A Allocation)**: $2,401.24
- **Total Requested Grant Amount**: $38,783.60

References

[ACLS18] Sebastian Angel, Hao Chen, Kim Laine, and Srinath T. V. Setty. PIR with compressed queries and amortized query processing. In *S&P*, 2018.

[BCP15] Elette Boyle, Kai-Min Chung, and Rafael Pass. Oblivious parallel ram. In *Theory of Cryptography Conference (TCC)*, 2015.

[BIM00] Amos Beimel, Yuval Ishai, and Tal Malkin. Reducing the servers computation in private information retrieval: Pir with preprocessing. In *CRYPTO*, pages 55–73, 2000.

[BS80] Jon Louis Bentley and James B Saxe. Decomposable searching problems i. static-to-dynamic transformation. *Journal of Algorithms*, 1(4):301–358, 1980.

[CCMS22] T.-H. Hubert Chan, Kai-Min Chung, Bruce M. Maggs, and Elaine Shi. Foundations of differentially oblivious algorithms. *J. ACM*, 69(4):27:1–27:49, 2022.

[CD24] Sofia Celi and Alex Davidson. Call me by my name: Simple, practical private information retrieval for keyword queries. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, page 4107–4121, 2024.

[CGKS95] Benny Chor, Oded Goldreich, Eyal Kushilevitz, and Madhu Sudan. Private information retrieval. In *FOCS*, 1995.

[CK20] Henry Corrigan-Gibbs and Dmitry Kogan. Private information retrieval with sublinear online time. In *EUROCRYPT*, 2020.

[DFD+21] Emma Dauterman, Vivian Fang, Ioannis Demertzis, Natacha Crooks, and Raluca Ada Popa. Snoopy: Surpassing the scalability bottleneck of oblivious storage. In *SOSP '21: ACM SIGOPS 28th Symposium on Operating Systems Principles, Virtual Event / Koblenz, Germany, October 26-29, 2021*, pages 655–671. ACM, 2021.

[DMWS12] John Demme, Robert Martin, Adam Waksman, and Simha Sethumadhavan. Side-channel vulnerability factor: A metric for measuring information leakage. In *International Symposium on Computer Architecture (ISCA)*, 2012.

[GWC+25] Tianyao Gu, Yilei Wang, Bingnan Chen, Afonso Tinoco, Elaine Shi, and Ke Yi. Efficient oblivious sorting and shuffling for hardware enclaves. In *Usenix Security*, 2025.

[GZS24] Ashrujit Ghoshal, Mingxun Zhou, and Elaine Shi. Efficient pre-processing pir without public-key cryptography. In *Eurocrypt*, 2024.

[GZSP25] Ashrujit Ghoshal, Mingxun Zhou, Elaine Shi, and Bo Peng. Pseudorandom functions with weak programming privacy and applications to private information retrieval. In *Eurocrypt*, 2025.

[HCS17] T.-H. Hubert Chan and Elaine Shi. Circuit opram: Unifying statistically and computationally secure orams and oprams. In *TCC*, 2017.

[HHCG+23] Alexandra Henzinger, Matthew M. Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. One server for the price of two: Simple and fast single-server private information retrieval. In *Usenix Security*, 2023.

[HPPY25] Alexander Hoover, Sarvar Patel, Giuseppe Persiano, and Kevin Yeo. Plinko: Single-server PIR with efficient updates via invertible PRFs. In *Eurocrypt*, 2025.

[KCG21] Dmitry Kogan and Henry Corrigan-Gibbs. Private blocklist lookups with checklist. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 875–892. USENIX Association, August 2021.

[LP23] Arthur Lazzaretti and Charalampos Papamanthou. Treepir: Sublinear-time and polylog-bandwidth private information retrieval from ddh. In *CRYPTO*, 2023.

[MW22] Samir Jordan Menon and David J. Wu. SPIRAL: Fast, high-rate single-server PIR via FHE composition. In *IEEE S&P*, 2022.

[ola] https://github.com/obliviouslabs/oram.

[RTSS09] Thomas Ristenpart, Eran Tromer, Hovav Shacham, and Stefan Savage. Hey, you, get off of my cloud: exploring information leakage in third-party compute clouds. In *ACM Conference on Computer and Communications Security (CCS)*, pages 199–212, New York, NY, USA, 2009. ACM.

[sig] Technology deep dive: Building a faster oram layer for enclaves. https://signal.org/blog/building-faster-oram/.

[SvDS+13] Emil Stefanov, Marten van Dijk, Elaine Shi, Christopher Fletcher, Ling Ren, Xiangyao Yu, and Srinivas Devadas. Path ORAM – an extremely simple oblivious ram protocol. In *CCS*, 2013.

[TGS23] Afonso Tinoco, Sixiang Gao, and Elaine Shi. Enigmap: External-memory oblivious map for secure enclaves. In *USENIX Security Symposium*, 2023.

[WCS15] Xiao Wang, T.-H. Hubert Chan, and Elaine Shi. Circuit ORAM: on tightness of the goldreich-ostrovsky lower bound. In *ACM CCS*, 2015.

[WNL+14] Xiao Shaun Wang, Kartik Nayak, Chang Liu, T-H. Hubert Chan, Elaine Shi, Emil Stefanov, and Yan Huang. Oblivious Data Structures. In *CCS*, 2014.

[WR25] Zhikun Wang and Ling Ren. Single-server client preprocessing PIR with tight space-time trade-off. In *Eurocrypt*, 2025.

[YNZ25] Sen Yang, Kartik Nayak, and Fan Zhang. Decentralization of ethereum's builder market, 2025.

[ZPSZ24] Mingxun Zhou, Andrew Park, Elaine Shi, and Wenting Zheng. Piano: Extremely simple, single-server pir with sublinear server computation. In *IEEE S& P*, 2024.

[ZSF25] Mingxun Zhou, Elaine Shi, and Giulia Fanti. Pacmann: Efficient private approximate nearest neighbor search. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.